

ARM7 Exception and Interrupt Handling

1 Operating Modes in ARM7

ARM7 supports multiple operating modes to safely handle normal program execution, interrupts, and exceptional conditions. Mode switching improves system protection and performance.

Operating Modes

- **User Mode** – Executes application programs (non-privileged)
- **FIQ Mode** – Handles fast, high-priority interrupts
- **IRQ Mode** – Handles normal interrupts
- **Supervisor Mode** – Executes OS and system-level code
- **Abort Mode** – Handles invalid memory access
- **Undefined Mode** – Handles unsupported instructions

All modes except User mode are **privileged modes**.

2 Register Banking in Exception Modes

Register banking allows ARM7 to provide separate physical registers for different modes, reducing the need to save and restore registers during interrupts.

Common Registers

- **R0–R6** are shared across all operating modes
- These registers are not banked

FIQ Mode Registers

- Registers **R7–R14** are banked in FIQ mode
- Provides private registers for fast interrupt handling
- Minimizes stack usage and latency

Registers in Other Exception Modes

- Each exception mode has its own:
 - **R13** – Stack Pointer (SP)
 - **R14** – Link Register (LR)
- Ensures separate stacks and return addresses

Register	User	IRQ	FIQ	Supervisor	Abort	Undefined
R0–R6	C	C	C	C	C	C
R7–R12	C	C	B	C	C	C
R13 (SP)	B	B	B	B	B	B
R14 (LR)	B	B	B	B	B	B

C = Common, B = Banked

3 CPSR and SPSR

Current Program Status Register (CPSR)

- Holds current processor state
- Stores condition flags (N, Z, C, V)
- Controls interrupt enable/disable
- Determines processor mode and instruction state

Saved Program Status Register (SPSR)

- Exists only in exception modes
- Stores old CPSR value on exception entry
- Used to restore processor state on exception return

CPSR is common to all modes, while each exception mode has its own SPSR.

4 Exception Handling Mechanism

ARM7 automatically performs a sequence of actions when an exception or interrupt occurs, ensuring safe and predictable system behavior.

Exception Entry Sequence

1. Save required user registers onto the stack
2. Switch processor to privileged exception mode

3. Save current PC into exception mode's Link Register
4. Copy CPSR into the exception mode's SPSR
5. Disable IRQ interrupts (and FIQ if applicable)
6. Load exception vector address into PC
7. Begin execution of exception handler

The exception handler restores registers and CPSR before returning to normal program execution.

5 Classification of Exceptions

ARM7 exceptions are classified based on their cause and relationship with instruction execution.

Types of Exceptions

- **Instruction-Caused Exceptions**
 - Software interrupt
 - Undefined instruction
 - Instruction or data abort
- **Exceptions as Side Effects of Instructions**
 - Data abort due to invalid memory access
 - Permission or page faults
- **Exceptions Not Related to Instruction Execution**
 - Reset
 - IRQ
 - FIQ

6 Supported Data Types

ARM7 supports flexible data formats to efficiently handle embedded and general- purpose computing tasks.

Data Sizes

- 1 byte (8-bit)
- 2 bytes (16-bit)
- 4 bytes (32-bit)

Data Representation

- Signed data
- Unsigned data

Operand Types

- Register operands
- Immediate operands

7 Quick Revision Summary

ARM7 achieves efficient and secure exception handling using multiple operating modes, register banking, CPSR/SPSR state management, and a structured exception entry mechanism.

- Multiple operating modes ensure protection and speed
- Register banking enables fast interrupt response
- CPSR and SPSR preserve processor state
- Exception handling is hardware-assisted
- Supports flexible data sizes and formats